

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
ФАКУЛЬТЕТ СИСТЕМ УПРАВЛЕНИЯ И РОБОТОТЕХНИКИ

РАЗРАБОТКА СИМУЛЯЦИОННОЙ СИСТЕМЫ
АВТОНОМНОГО ДВИЖЕНИЯ И СТЫКОВКИ
ДИФФЕРЕНЦИАЛЬНОГО МОБИЛЬНОГО РОБОТА В ROS 2

Аннотация – В работе разработана симуляционная система автономного движения и стыковки дифференциального мобильного робота. Проект реализован на ROS 2 и Gazebo Sim. Робот движется по белому треку, использует лидар для простой реакции на препятствия, распознает ArUco-маркеры с помощью OpenCV и выполняет финальную стыковку по маркеру док-станции. Для повышения устойчивости оценки положения маркера используется фильтр Калмана, а для формирования управляющих скоростей применяется PD-регулятор.

Ключевые слова: ROS 2, Gazebo, differential drive, ArUco, OpenCV, Kalman filter, PD controller, autonomous docking.

Выполнили

Май Хоанг Куан

Ису: 407882,

*ТГ: @quanmh_25,
itmoquanmh@gmail.com*

Проверил

Живицкий Андрей Юрьевич

Содержание

1	Введение	2
2	Архитектура системы	2
2.1	Структура проекта	2
2.2	Запуск системы	2
2.3	Сенсоры и приводы	3
2.4	Ключевые ROS 2 топики	3
3	Алгоритмы управления	3
3.1	Общая логика миссии	3
3.2	Слежение за белым треком	4
3.3	Обход препятствий по лидару	4
3.4	Распознавание ArUco-маркеров	5
3.5	Фильтр Калмана	5
3.6	Формирование waypoint	5
3.7	PD-регулятор	5
3.8	Конечный автомат docking	6
4	Реализация	6
4.1	Основные файлы	6
4.2	Docker-окружение	7
5	Запуск и тестирование	7
5.1	Сборка workspace	7
5.2	Запуск полной системы	7
5.3	Полезные команды проверки	7
6	Результаты	8
7	Обсуждение	9
8	Заключение	9

1 Введение

Автономная стыковка является важной задачей мобильной робототехники. Она используется в роботах-доставщиках, складских платформах, сервисных роботах и системах автоматической зарядки. Для выполнения такой задачи робот должен воспринимать окружающую среду, двигаться по маршруту, распознавать ориентиры и точно подъезжать к целевой позиции.

Цель данного проекта состоит в разработке симуляционной системы, в которой дифференциальный робот:

- запускается в мире Gazebo;
- движется по белому треку на основе изображения с камеры;
- использует 2D-лидар для обнаружения близких препятствий;
- распознает ArUco-маркеры;
- после обнаружения маркера док-станции выполняет автоматическую стыковку.

Проект был выполнен как ROS 2 workspace с двумя основными пакетами: `robot_scene` и `robot_controller`. Для воспроизводимого запуска также подготовлены `Dockerfile` и `docker-compose.yaml`.

2 Архитектура системы

2.1 Структура проекта

Проект разделен на пакет сцены и пакет управления. Такой подход упрощает поддержку: описание робота, мира и мостов Gazebo отделено от алгоритмов автономного движения.

Пакет	Назначение
<code>robot_scene</code>	Хасро/URDF-модель робота, мир Gazebo, модели ArUco, конфигурация ROS-Gazebo bridge, <code>ros2_control</code> , RViz.
<code>robot_controller</code>	Основная логика автономного движения: обработка камеры, обработка лидара, распознавание ArUco, фильтр Калмана, PD-регулятор и конечный автомат миссии.

Таблица 1: Основные пакеты проекта

2.2 Запуск системы

Главный launch-файл проекта — `robot_controller/launch/run.launch.py`. Он подключает launch-файл сцены `robot_scene/launch/bringup.launch.py`, запускает окно `rqt_image_view` для просмотра отладочного изображения и с задержкой запускает узел `control_docking`.

Launch-файл сцены выполняет следующие действия:

- запускает Gazebo Sim с миром `map.sdf`;

- публикует описание робота через `robot_state_publisher`;
- создает робота в позиции `x=7`, `y=1`, `z=0.15`;
- запускает `ros_gz_bridge`;
- запускает `joint_state_broadcaster` и `diff_drive_controller`;
- открывает RViz с подготовленной конфигурацией.

2.3 Сенсоры и приводы

Модель робота является дифференциальной платформой. Управление колесами выполняется через `ros2_control` и `diff_drive_controller`. В конфигурации контроллера используются колеса `wheel_left_joint` и `wheel_right_joint`, радиус колеса равен 0.1, расстояние между колесами — 0.65. Команды скорости принимаются в формате `TwistStamped`.

Для восприятия среды используются:

- камера, публикующая изображение в `/camera/image_raw`;
- 2D-лидар, публикующий скан в `/diff_drive/scan`;
- статические и динамические TF-фреймы для визуализации робота, маркера и целевой точки.

2.4 Ключевые ROS 2 топики

Топик	Тип сообщения	Описание
<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	Изображение с камеры робота.
<code>/camera/camera_info</code>	<code>sensor_msgs/CameraInfo</code>	Параметры камеры.
<code>/camera/image_debug</code>	<code>sensor_msgs/Image</code>	Изображение после обработки, используется для отладки ArUco.
<code>/diff_drive/scan</code>	<code>sensor_msgs/LaserScan</code>	Данные 2D-лидара.
<code>/diff_drive_controller/cmd_vel</code>	<code>geometry_msgs/TwistStamped</code>	Команды линейной и угловой скорости.
<code>/tf, /tf_static</code>	<code>tf2_msgs/TFMessage</code>	Преобразования координат робота и отладочных фреймов.

3 Алгоритмы управления

3.1 Общая логика миссии

Алгоритм реализован в узле `control_docking`. Узел подписывается на изображение камеры и данные лидара, публикует команды скорости и отладочное изображение, а также транслирует TF-фреймы отфильтрованного маркера и целевой точки.

Миссия робота состоит из следующих этапов:

1. движение по белому треку;
2. реакция на близкие стены или препятствия по лидару;
3. обнаружение ArUco ID 3 и подготовка к повороту;
4. выполнение поворота к маршруту док-станции;
5. обнаружение ArUco ID 23;
6. оценка положения док-станции, фильтрация и стыковка;
7. остановка после завершения docking.

3.2 Слежение за белым треком

Для движения по треку используется нижняя часть изображения камеры. В коде задается `roi_height_ratio = 0.3`, то есть анализируется нижняя область кадра. Далее изображение переводится в оттенки серого, сглаживается фильтром Гаусса и бинаризуется адаптивным порогом.

После бинаризации находятся контуры. Самый большой контур с площадью больше `min_hole = 300` считается найденным участком трека. По центру масс контура определяется положение трека:

- LEFT, если центр находится в левой трети изображения;
- CENTER, если центр находится в центральной трети;
- RIGHT, если центр находится в правой трети.

На основе этого положения робот выбирает направление поворота. В режиме `FOLLOW_TRACK` при обнаружении трека робот поворачивает, чтобы удерживать маршрут в поле зрения камеры.

3.3 Обход препятствий по лидару

Данные лидара делятся на три угловых сектора:

- передний сектор: от -20° до 20° ;
- левый сектор: от 45° до 90° ;
- правый сектор: от -90° до -45° .

Некорректные значения дальности заменяются на `NaN`, после чего для каждого сектора вычисляется минимальная дистанция. Если перед роботом расстояние меньше `stop_dist = 1.0`, робот переходит в режим `AVOID`. Когда расстояние становится больше `clear_dist = 1.4`, робот возвращается в режим `GO`. Направление обхода выбирается по более свободной стороне.

3.4 Распознавание ArUco-маркеров

Для распознавания маркеров используется OpenCV ArUco dictionary DICT_4X4_250. В проекте используются два важных маркера:

ID маркера	Назначение
3	Сигнал для подготовки и выполнения поворота к маршруту док-станции.
23	Маркер док-станции, по которому выполняется финальное позиционирование.

Таблица 3: ArUco-маркеры в проекте

Если обнаружен маркер ID 3, вычисляется площадь его контура. Пока площадь меньше 8500, робот находится в состоянии подготовки к повороту. Когда маркер становится достаточно крупным в кадре, миссия переходит в режим выполнения поворота.

Если обнаружен маркер ID 23, робот переходит в режим DOCKING_23. Для маркера оценивается pose через `estimatePoseSingleMarkers`. Из вектора поворота получается матрица вращения, затем вычисляется yaw. Состояние маркера представляется в виде:

$$x = t_z, \quad y = -t_x, \quad \psi = yaw.$$

3.5 Фильтр Калмана

Измерения положения ArUco-маркера могут шуметь из-за дискретности изображения, перспективы и ошибок детектора. Поэтому для маркера ID 23 используется фильтр Калмана с состоянием:

$$X = [x \quad y \quad \psi]^T.$$

В проекте используются матрицы:

$$F = I, \quad B = -I, \quad H = I.$$

Шаг предсказания учитывает линейную и угловую скорость робота. Шаг коррекции обновляет оценку по измерению ArUco. Угол yaw нормализуется в диапазон от $-\pi$ до π . Отфильтрованная pose публикуется в TF как `filter_aruco_link`.

3.6 Формирование waypoint

После фильтрации положения маркера создается промежуточная целевая точка перед док-станцией:

$$x_{wp} = x + 0.6 \cos(\psi), \quad y_{wp} = y + 0.6 \sin(\psi), \quad \psi_{wp} = \psi.$$

Эта точка публикуется как TF-фрейм `aruco_waypoint1`. Использование waypoint позволяет роботу сначала выровняться перед станцией, а затем подъехать к самому маркеру.

3.7 PD-регулятор

Для вычисления управляющего воздействия используется PD-регулятор:

$$u = K_p e + K_d \frac{e - e_{prev}}{\Delta t}.$$

В проекте заданы коэффициенты:

$$K_p = [0.25, 0.25, 0.25], \quad K_d = [0.01, 0.01, 0.01].$$

Выход регулятора преобразуется в линейную и угловую скорость. Команды ограничиваются значениями:

$$|v| \leq 0.3, \quad |\omega| \leq 0.25.$$

Это ограничение делает финальную стыковку более плавной и уменьшает риск резких движений рядом с док-станцией.

3.8 Конечный автомат docking

Финальный docking реализован как последовательность состояний:

Состояние	Действие
DETECTION	Накопление детекций маркера ID 23. После 120 обнаружений робот переходит к waypoint.
WAYPOINT	Поворот робота для выравнивания по промежуточной точке.
COME_TO_WAYPOINT	Движение к промежуточной точке перед док-станцией.
STATION	Поворот и выравнивание относительно самой станции.
COME_TO_STATION	Финальное движение к маркеру ID 23.
END / DOCKED	Остановка робота после достижения порогов по линейной и боковой ошибке.

Таблица 4: Состояния финальной стыковки

4 Реализация

4.1 Основные файлы

Файл	Назначение
control_docking.py	Главный ROS 2 node: подписки, обработка сенсоров, state machine, публикация скорости.
aruco_detector.py	Класс для обнаружения ArUco и оценки pose маркера.
kalman_filter.py	Реализация фильтра Калмана для состояния [x, y, yaw].
pd_controller.py	Реализация PD-регулятора.
utils.py	Нормализация угла и перевод координат из декартовой формы в полярную.
robot_model.xacro	Описание робота, колес, камеры, лидара и ros2_control.
diff_control.yaml	Параметры diff_drive_controller.
ros_gz_bridge.yaml	Мост Gazebo topics в ROS 2 topics.

Таблица 5: Ключевые файлы проекта

4.2 Docker-окружение

Для удобного запуска подготовлен Docker. Он позволяет запускать проект в повторяемом окружении с ROS 2, Gazebo, RViz, OpenCV и colcon. Основной сценарий работы:

```
1 cd diffdrive_aruco_docking
2 export GID=$(id -g)
3 xhost +local:docker
4 docker compose up -d --build
5 docker compose exec ros bash
```

Листинг 1: Первый запуск через Docker

Внутри контейнера выполняется сборка и запуск:

```
1 colcon build --symlink-install
2 source install/setup.bash
3 ros2 launch robot_controller run.launch.py
```

Листинг 2: Сборка и запуск внутри контейнера

После выхода из shell контейнера командой `exit` сам контейнер, запущенный через `docker compose up -d`, продолжает работать. Для полной остановки используется:

```
1 docker compose down
```

Листинг 3: Остановка Docker-контейнера

5 Запуск и тестирование

5.1 Сборка workspace

```
1 cd workspace
2 colcon build --symlink-install
3 source install/setup.bash
```

Листинг 4: Сборка проекта

5.2 Запуск полной системы

```
1 ros2 launch robot_controller run.launch.py
```

Листинг 5: Запуск симуляции и автономного контроллера

Этот launch-файл запускает Gazebo, RViz, мост Gazebo-ROS, контроллер колес, окно просмотра изображения и узел автономной стыковки.

5.3 Полезные команды проверки

```
1 ros2 topic list
2 ros2 control list_controllers
3 ros2 topic echo /camera/camera_info --once
4 ros2 topic echo /diff_drive/scan --once
5 ros2 topic echo /diff_drive_controller/cmd_vel
```

Листинг 6: Проверка ROS 2 topics и controllers

Для визуальной проверки детекции ArUco используется:

```
1 rqt_image_view
```

Листинг 7: Просмотр отладочного изображения

В окне необходимо выбрать топик `/camera/image_debug`.

6 Результаты

В результате работы был создан полный симуляционный pipeline для автономной стыковки робота. Реализованы модель робота, окружение Gazebo, ROS-Gazebo bridge, контроллер дифференциальной платформы, обработка камеры, обработка лидара и алгоритм docking.

Задача	Результат	Комментарий
Запуск мира Gazebo и модели робота	Выполнено	Мир <code>map.sdf</code> , модели ArUco и робот загружаются через <code>launch</code> .
Управление дифференциальным роботом	Выполнено	Используется <code>ros2_control</code> и <code>diff_drive_controller</code> .
Получение изображения камеры	Выполнено	Топик <code>/camera/image_raw</code> передается из Gazebo в ROS 2.
Слежение за белым треком	Выполнено	Используется ROI, <code>adaptive threshold</code> и анализ контуров.
Реакция на препятствия по лидару	Выполнено	Используются передний, левый и правый сектора LaserScan.
Обнаружение ArUco ID 3	Выполнено	Маркер используется для перехода к повороту.
Обнаружение ArUco ID 23	Выполнено	Маркер используется как док-станция.
Фильтрация pose маркера	Выполнено	Реализован фильтр Калмана для $[x, y, yaw]$.
Docking по waypoint и маркеру станции	Выполнено	Реализован state machine и PD-регулятор.

Таблица 6: Итоги реализации

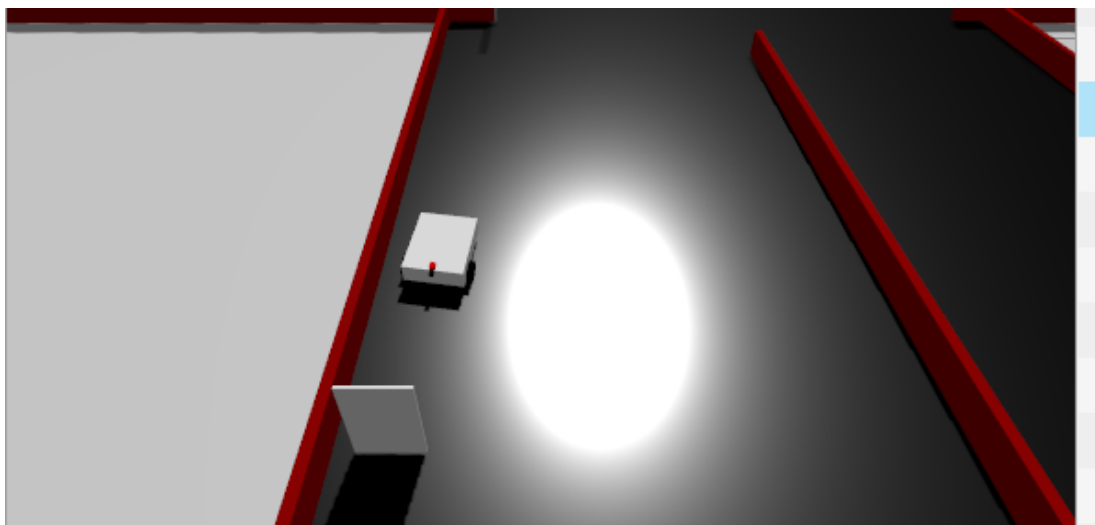


Рис. 1: Демонстрация проекта в симуляции

7 Обсуждение

Главным преимуществом проекта является модульная структура. Сцена, модель робота и конфигурации запуска находятся отдельно от алгоритмов управления. Это упрощает отладку: можно отдельно запускать только сцену через `robot_scene/bringup.launch.py` или полную систему через `robot_controller/run.launch.py`.

Использование ArUco-маркеров удобно для симуляции и прототипирования, потому что маркеры легко детектируются и позволяют оценить pose относительно камеры. Фильтр Калмана повышает устойчивость оценки положения док-станции, а PD-регулятор делает финальное движение более плавным.

Ограничения текущей реализации:

- алгоритм слежения за белой линией зависит от качества изображения и освещения;
- параметры PD-регулятора подобраны вручную;
- логика обхода препятствий является простой реактивной схемой;
- docking проверялся в симуляционной среде, а для реального робота потребуются калибровка камеры и настройка параметров движения.

В дальнейшем проект можно улучшить следующими способами:

- добавить более устойчивую обработку линии с учетом перспективного преобразования;
- использовать PID или MPC для финального docking;
- добавить автоматическую калибровку камеры;
- расширить state machine для обработки потери маркера;
- перенести систему с симуляции на физического робота.

8 Заключение

В рамках проекта была реализована система автономной стыковки дифференциального робота в ROS 2 и Gazebo. Робот способен запускаться в симуляционной среде, получать данные камеры и лидара, следовать по маршруту, распознавать ArUco-маркеры и выполнять финальную стыковку по маркеру док-станции.

Основной результат работы — готовый ROS 2 pipeline, который объединяет Gazebo Sim, `ros2_control`, OpenCV ArUco, фильтр Калмана, PD-регулятор и конечный автомат управления. Проект также содержит Docker-окружение, что делает запуск более воспроизводимым и удобным для дальнейшего развития.

Список литературы

- [1] ROS 2 Documentation. <https://docs.ros.org/>
- [2] Gazebo Sim Documentation. <https://gazebo-sim.org/docs>
- [3] OpenCV ArUco module. <https://docs.opencv.org/>
- [4] `ros2_control` Documentation. <https://control.ros.org/>